

OPTIMISING TRAFFIC LIGHT DECISIONS TO REDUCE WAITING TIME AND CONGESTION

Reinforcement Learning, CA2 group project

HIGHLIGHTS

- A custom simulation was implemented using python and Gymnasium
- Two RL algorithms- Q-learning and Monte Carlo were implemented
- The system learns optimal signal switching strategies to reduce waiting time

KEYWORDS

Reinforcement learning, Traffic Signal Control, Q-learning, Monte Carlo, Markov Decision Process

ABSTRACT

Traffic congestion at road intersections is a common problem that leads to increased traffic flow and waiting time. This study explores the applications of using reinforcement learning (RL) to develop an adaptive traffic control system.

A custom simulation environment represents two direction intersections North-South and East-West. The RL agents observe the number of vehicles waiting in each direction and select an action to control which direction receives the green signal. The goal is to minimise the number of vehicles waiting in both directions.

Two reinforcement learning algorithms are implemented, Monte Carlo and Q-Learning. They were evaluated and the results demonstrate that Q-learning achieves faster convergence during training, whereas Monte Carlo reliable value estimation through episode based learning.

1 INTRODUCTION

Traffic congestion is one of the most pressing challenges in the transportation system. Increasing vehicle numbers and limited infrastructure often result in inefficient traffic flow, especially when there are signals. Traditionally traffic lights operate in a predefined timing schedules which cannot respond dynamically to changing traffic conditions

Recent advancements in Artificial Intelligence have introduced Reinforcement learning which is an approach for decision making problems in a dynamic environment. RL enables an agent to learn optimal strategies through interaction with environment to maximize rewards

Traffic control is well suited for RL because it naturally involves an agent which was used as traffic controller , an environment which is road intersection , states which are traffic conditions , actions which are signal switching and rewards which is traffic efficiency

The project proposes an RL based adaptive traffic signal system capable of learning traffic patterns and optimising signal decisions over time . Two reinforcement learning algorithms are implemented and evaluated which are Q- learning and Monte Carlo . The goal is to provide better performance in reducing traffic congestion at an intersection.

1.1 IMPROVEMENTS OF USING REINFORCEMENT LEARNING

Reinforcement learning can bring an enhancement to traffic signals by enabling adaptive and data-driven signal control. Unlike fixed timing, RL agents can continuously learn from traffic conditions and adjust decisions dynamically . Key improvements include adaptive traffic signal timing , reduced waiting time , improved traffic flow efficiency and adaptability to respond to unpredictable traffic patterns. This is used because RL learns optimal policies through interaction with environment , it is well suited for complex and dynamic traffic management problems

2 METHODOLOGY

2.1 PROBLEM FORMULATION

The traffic signal problem is formulated as an RL task where an agent learns an optimal policy task through interaction with a simulated traffic environment . The objective is to minimise vehicle congestion and waiting time at an intersection by selecting appropriate traffic signal actions . The problems is modelled as Markov Decision Problem which consists of agents, environments, state, action and rewards

- **Agent**
Traffic Signal Controller responsible for deciding which traffic direction would receive the green signal
- **Environment**
A simulated intersection with two traffic directions, which are North-South(NS) and East-West(EW)
The environment models several aspects of of traffic behavior including
 - Vehicle arrival
 - Vehicle departure
 - Traffic capacity limits
 - Random Traffic variation
- **State**
A state represents the current traffic in each direction . It is represented by the number of cars waiting in each direction . In order to simplify the problem and reduce complexity ,the traffic capacity for each direction is limited to 0 to 3 cars in the North -South(NS) lane and 0 to 5 vehicles in the East-West(EW) lane.

- **Action**

Action is selecting which direction receives a green signal. For action space, the agents can choose between 2 actions: 0 for EW direction and 1 for NS direction. Only one direction receives a green signal at a time.

- **Reward**

A reward signal is designed to encourage the agent to minimise traffic congestion. In this project the reward is defined as the negative of the total number of cars waiting in both directions.

2.2 TRAFFIC ENVIRONMENT SIMULATION

A custom traffic simulation environment was implemented in Python using the Gymnasium framework. The environment models simplified traffic behaviour through three key mechanisms: vehicle arrivals, vehicle departures, and road capacity constraints.

- **Vehicle Arrival**

At every time step, 2 vehicles arrive in each direction. The number of incoming vehicles is generated within a small range to simulate unpredictable traffic flow.

- **Vehicle Departure**

When the traffic signal turns green for a specific direction, vehicles in that direction are allowed to pass through the intersection. Only 2 of vehicles are removed from the queue depending on the current traffic conditions.

- **Capacity Constraints**

To maintain realistic behaviour, the number of vehicles in each direction is restricted to predefined limits. Vehicle counts cannot fall below zero or exceed the maximum road capacity.

These mechanisms allow the simulation to represent dynamic traffic patterns while remaining computationally efficient for reinforcement learning training.

2.3 REINFORCEMENT LEARNING ALGORITHM

To solve this problem two reinforcement algorithms were implemented and compared- Q- learning and Monte Carlo. Both algorithms aim to estimate the action value function $Q(s,a)$, which represents the cumulative reward obtained by taking the action a in state s and following the learning policy hereafter. After interaction with the environment, they differ in how they update their value estimates.

2.3.1 Q-LEARNING

Q- learning is a learning algorithm that updates the value of state-action pairs after every interaction with the environment. The algorithm maintains a Q -table that scores expected reward for each state-action pair.

The Q-value is updated using the Bellman equation:

$$Q(s,a) = Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where

- α represents the learning rate
- γ represents the discount factor
- r represents the reward received after performing action a
- s' represents the next state

This updated rule allows the algorithm to gradually improve estimates of optimal actions. This is because Q-learning values update values after every step, it can learn efficiently from each step.

2.3.2 MONTE-CARLO

Monte Carlo reinforcement learning estimates value functions based on complete episode experience rather than individual steps. Instead of updating the value function after each step, the algorithm waits after every action, the algorithm waits until the entire episode has finished and then calculates the total return obtained after the episode.

The return G represents cumulative discounted reward obtained from a particular step until end of episode

$$G = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

This study used first visit Monte Carlo, where each state action pair is updated only the first time it appears on the episode.

2.3.3 EXPLORATION STRATEGY

Both the reinforcement learning algorithm uses an epsilon greedy policy to balance exploration and exploitation. During training With probability ϵ , the agent selects a random action (exploration), With probability $1-\epsilon$, the agent selects the action with the highest estimated Q-value (exploitation).

At the beginning of training, the exploration rate is set to high to encourage the agent to explore different traffic control strategies.

2.3.4 TRAINING PROCEDURE

The training procedure involves repeated interaction between agent and traffic environment over a series of episodes

The training procedure involves

- Initializing Q- tables with zeros
- Reset the environment to generate initial traffic
- Observe the current state of intersection
- Select an action using epsilon greedy strategy
- Apply selected action of environment
- Observe next state and reward generated by environment
- Update the Q- values with monte carlo and Q- learning algorithms
- Repeat until process ends
- Store the reward got during episode

2.3.5 HYPERPARAMETER SETTINGS

Several hyperparameters control the behaviour of reinforcement learning algorithms

The following configuration was used

- Learning rate:0.1
- Discount Factor:0.9
- Initial exploration rate:1.0
- Exploration decay:0.995
- Minimum exploration rate:0.01
- Number of training episodes:5000

3 PERFORMANCE EVALUATION

The performance was evaluated based on reward based metrics and convergence analysis.

The primary evaluation metric was total reward per episode which represents the level of congestion experienced during training. Increased reward over time indicates the agent is learning to reduce waiting time.

Additionally the learned policies from Q-learning and Monte-Carlo were compared to determine which algorithm provided better performance in controlling traffic signals.

3.1 RESULTS

After implementing the reinforcement learning models, our team evaluated their performance by analysing the reward curves produced during training. The reward value reflects how efficiently the agent controls the traffic signals at the simulated intersection. In this environment, the reward is calculated as the negative number of cars waiting in both directions. Therefore, higher reward values indicate better traffic management because fewer vehicles are waiting at the intersection.

3.1.1 Q-LEARNING RESULTS

The first set of results shows the reward curve obtained during training with the Q-Learning algorithm.

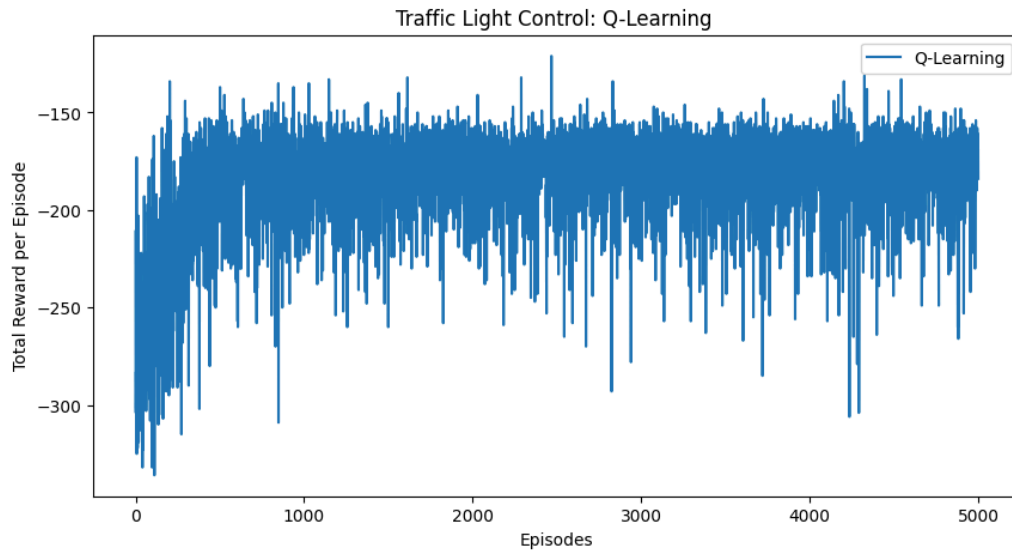


Figure 1: Reward curve during training using the Q-Learning algorithm

In the early episodes, the reward values fluctuate significantly. This behaviour occurs because the agent initially has no knowledge of the environment and therefore explores different actions randomly. As a result, traffic signals are sometimes assigned inefficiently, which leads to higher congestion and lower rewards. However, as the number of training episodes increases, the reward values begin to stabilise and show gradual improvement. This suggests that the agent is learning from previous experiences and gradually identifying better signal switching strategies.

3.1.2 MONTE CARLO RESULTS

The set of results presents the reward curve for the Monte Carlo algorithm.

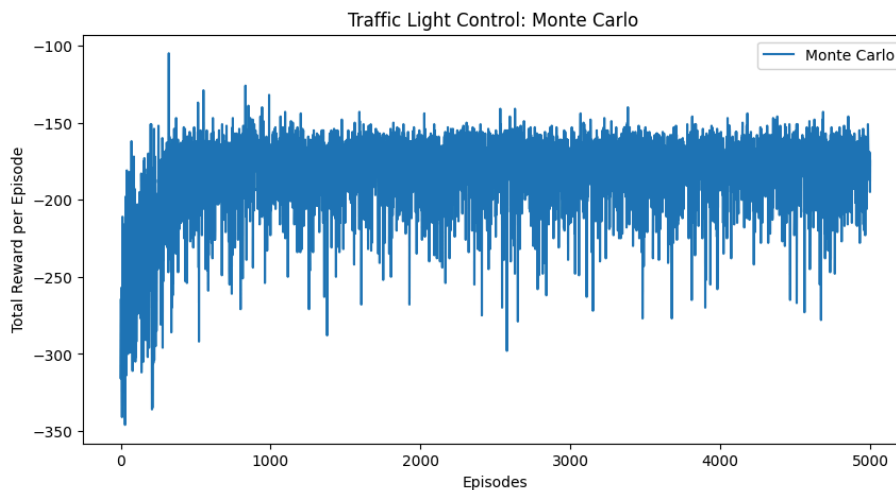


Figure 2: Reward curve during training using the Monte Carlo algorithm

Similar to Q-Learning, the early training episodes show high variability in reward values due to exploration. Since Monte Carlo learning updates its value estimates only after the completion of an episode, the learning process appears slower and the reward curve shows more fluctuations throughout training. Nevertheless, over time the model begins to identify better decisions and the reward values improve compared to the initial training stage.

3.1.3 ALGORITHM COMPARISON

A comparison of the two algorithms indicates that Q-Learning generally produces more stable reward patterns than the Monte Carlo method.

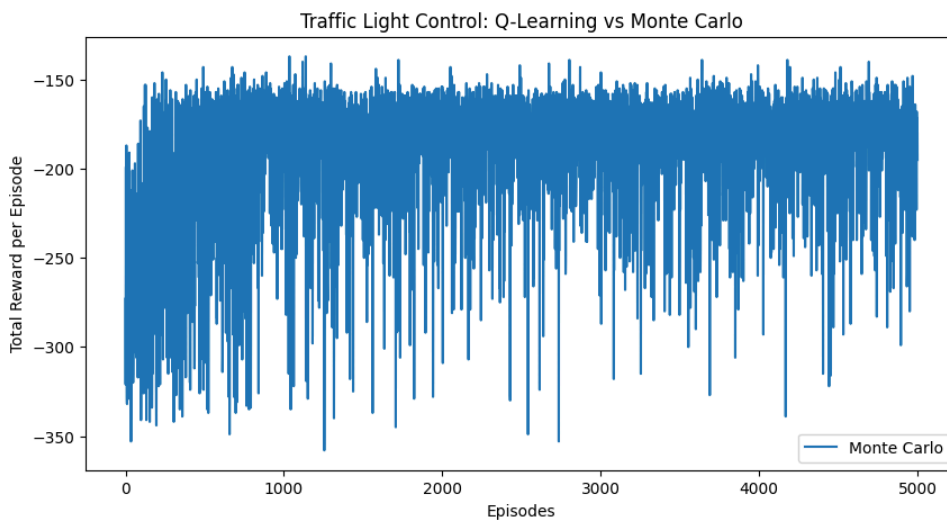


Figure 3:

Performance comparison between Q-Learning and Monte Carlo

Because Q-Learning updates its Q values after every interaction with the environment, it can adapt more quickly to changes in traffic conditions. In contrast, Monte Carlo learning relies on episode based updates, which results in slower policy improvement and greater variability in rewards. Despite these differences, both algorithms demonstrate the ability to learn adaptive traffic signal control strategies over repeated training episodes.

Overall, the results indicate that reinforcement learning can gradually improve decision making for traffic signal control. Through repeated interaction with the environment, the agents learn to prioritise the direction with higher traffic demand, which helps reduce congestion in the simulated intersection.

3.2 IMPACT OF HYPERPARAMETER CONFIGURATION ON PERFORMANCE

To further evaluate the behaviour of the reinforcement learning models, additional experiments were conducted by modifying several hyperparameters and observing how these changes influenced learning performance. t.

In the first configuration, the learning rate was set to 0.2, the discount factor to 0.3, the exploration rate started at 1.0, and the exploration decay rate was 0.456. The number of training episodes was limited to 500. Under these settings, both algorithms produced highly fluctuating reward curves across the training episodes. The reward values frequently ranged between approximately -150 and -320 , indicating unstable learning behaviour. Because exploration decreases quickly, the agent stops trying alternative actions early in the training process, which limits its ability to discover improved traffic control policies.

In the second configuration, the learning rate was increased to 0.3, the discount factor to 0.45, and the exploration rate was initially set to 1.5 with a decay rate of 0.564. The minimum exploration threshold was reduced to 0.02 while maintaining the same number of training episodes. Under this configuration, the reward curves again showed significant variability across episodes. Although slightly higher reward values were occasionally observed, the overall learning pattern still lacked clear convergence. This suggests that the modified parameters did not significantly improve the learning stability of the models.

A comparison of the learned policies also reveals differences between the two algorithms. The Q-Learning policy matrix indicates that the agent tends to assign green signals more frequently to the North–South direction in certain traffic states, while still allowing East–West traffic in others. In contrast, the Monte Carlo policy tends to favour East–West traffic in more states, which may lead to less balanced traffic control decisions. These differences arise because Q-Learning updates its value estimates at every step, whereas Monte Carlo learning relies on complete episode returns and therefore updates its estimates less frequently.

Overall, the hyperparameter comparison demonstrates that reinforcement learning performance is highly sensitive to parameter selection. Large learning rates, small discount factors, and aggressive exploration decay can cause unstable reward behaviour.

Experiment (EXP)	Learning Rate (α)	Discount Factor (γ)	Initial Exploration (ϵ)	Epsilon Decay	Minimum Epsilon	Episodes	Observed Behaviour
EXP 1	0.20	0.30	1.0	0.456	0.03	500	Reward values fluctuates heavily
EXP 2	0.30	0.45	0.5	0.564	0.02	500	Rewards remain unstable

4 DISCUSSION

From the results obtained in this project, our group observed that reinforcement learning can be applied to traffic signal control to help improve traffic flow at an intersection. Both Q-Learning and Monte Carlo algorithms were able to learn useful traffic signal strategies through repeated interaction with the simulated environment. Although the learning process was not completely stable throughout training, the reward curves show that the agents gradually improved their performance over time as they gained more experience with the environment.

One important observation from our experiments was the difference in learning behaviour between the two algorithms. Q-Learning appeared to learn faster and showed slightly more stable reward patterns during training. This may be because the algorithm updates its Q values after every interaction with the environment, which allows it to adjust its decisions more quickly. On the other hand, the Monte Carlo method updates its value estimates only after a full episode has been completed. Because of this, its learning process was slower and the reward curve showed more fluctuations.

Looking at the results from a practical point of view, our findings suggest that reinforcement learning could be useful for developing adaptive traffic management systems. In many real-world situations, traffic lights follow fixed timing schedules that do not change according to the actual traffic conditions. In contrast, an RL-based system can observe the traffic situation and adjust signal decisions dynamically. However, implementing such systems in real environments would require more detailed simulations, larger datasets, and more realistic traffic modelling.

Another aspect we noticed during training was the balance between exploration and exploitation. The epsilon-greedy strategy allowed the agent to explore different actions while also learning from previous experiences. While this exploration is necessary for learning better strategies, it sometimes causes the agent to choose inefficient actions. This behaviour explains why the reward curves occasionally fluctuate even after the agent has started learning.

Finally, it is important to recognise that the simulation used in this project represents a simplified traffic environment. The model only considers a basic intersection with limited traffic states. In reality, traffic systems are far more complex and may include multiple intersections, pedestrian crossings, and unpredictable traffic patterns. Expanding reinforcement learning approaches to such large scale environments would require more advanced algorithms and additional computational resources.

4.1 CHALLENGES FACED

During the development of this project, we faced several challenges while implementing the reinforcement learning models and designing the simulation environment. One of the main challenges was simplifying the traffic system while still keeping the environment meaningful for training the RL agents. Real-world traffic systems are complex and include many variables, but for this project we had to limit the environment to two directions in order to make the learning problem manageable.

Another challenge involved designing the reward function. Since the agent learns based on rewards, defining a reward that correctly represents traffic congestion requires careful consideration. Initially, it was not clear how to represent traffic efficiency through rewards. After several attempts, the reward was defined as the negative number of cars waiting in both directions, which encouraged the agent to minimise congestion and reduce waiting time.

Training stability was also a challenge during the early stages of the project. The reward curves showed large fluctuations because the agent was exploring different actions using the epsilon-greedy policy. At first it was difficult to determine whether the agent was actually learning or simply acting randomly. Increasing the number of training episodes and analysing the reward trends helped us better understand the learning behaviour.

In addition, comparing two reinforcement learning algorithms required careful analysis of their learning behaviour. Since Q-Learning updates its values after each step while Monte Carlo updates after full episodes, their learning patterns were different. Understanding and explaining these differences required additional experimentation and analysis of the reward curves.

If additional time and resources were available, several improvements could further enhance the system. One potential improvement involves refining the reward function to better capture traffic efficiency, such as incorporating penalties for long queue durations or prioritising high-density lanes. In addition, more advanced exploration strategies could be applied to balance exploration and exploitation more effectively during training. Future work could also explore deep reinforcement learning approaches and investigate transferring policies learned in simulation to real-world traffic environments.

5 MODEL DEPLOYMENT AND PRACTICAL VALIDATION

After completing the training and evaluation stage of the reinforcement learning models, the next phase of the project focused on deployment and practical validation. The objective of deployment was to convert the trained traffic signal controller into a usable software application that demonstrates how reinforcement learning can operate in a real-world style environment. Based on the implementation in the project code, the system was deployed using the Flask web framework.

Flask was selected because it is a lightweight Python framework that allows machine learning models to be integrated into web applications with minimal complexity. By using Flask, the trained Q-learning model was transformed into an interactive browser-based application where users can enter traffic conditions and receive signal control decisions instantly.

The deployed application simulates a two-direction road intersection involving North–South and East–West traffic movement. Users provide the current queue lengths for both directions, and the reinforcement learning agent determines which direction should receive the green signal. This demonstrates how the learned policy can be used outside the training environment.

Deployment is an essential part of reinforcement learning projects because successful training alone does not guarantee practical usefulness. A model must also be able to respond quickly, process live inputs, and provide understandable outputs. Through the Flask application, the project validates that the trained traffic control system can function as a decision-support tool.

The deployment stage also satisfies the artifact demonstration requirement of the assessment by showing a working software solution rather than only theoretical results. Users can directly interact with the system, test different traffic scenarios, and observe how reinforcement learning decisions change according to queue conditions.

Overall, the deployment proves that reinforcement learning can move beyond experimental notebooks and be integrated into practical web-based traffic management tools.

5.1 SYSTEM ARCHITECTURE

The deployed traffic signal control system follows a modular software architecture composed of multiple interconnected components. This design improves readability, maintainability, and scalability of the project.

The first component is the User Interface Layer. This layer is built using HTML rendered through Flask templates. It provides a browser-based form where users can input the number of waiting vehicles in both traffic directions. The interface also displays the final signal decision predicted by the reinforcement learning model.

The second component is the Application Controller Layer, managed by Flask routes. This layer receives user requests, processes form data, communicates with the trained model, and sends prediction results back to the browser. It acts as the connection between frontend interaction and backend intelligence.

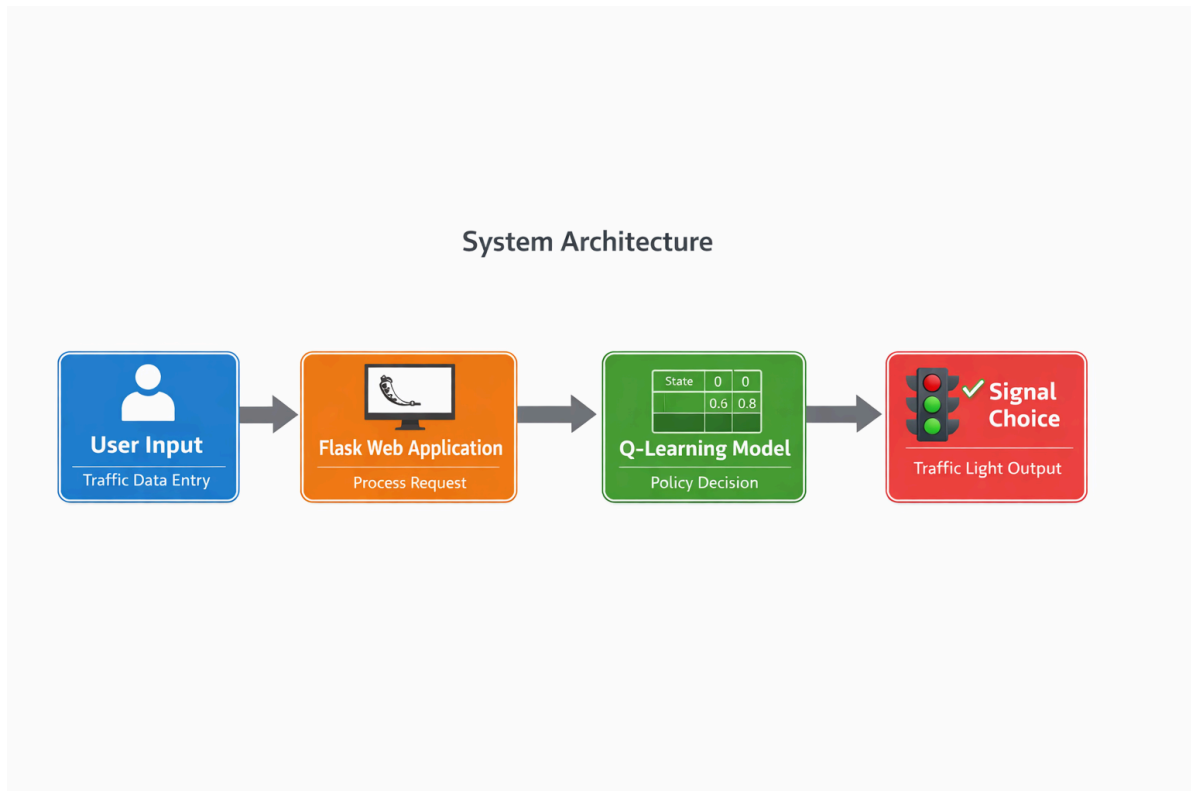
The third component is the Reinforcement Learning Decision Engine. This module contains the trained Q-table produced during the learning phase. Based on the entered traffic state, the model selects the action with the highest Q-value.

The fourth component is the Simulation Logic Layer, which defines traffic states, possible actions, reward behaviour, and state transitions used during training. Although training occurs separately, the learned behaviour is embedded into deployment.

The architecture workflow operates as follows:

1. The user opens the Flask web application.
2. Users enter queue lengths for North–South and East–West lanes.
3. Flask receives the input values.
4. The current state is matched with the trained Q-table.
5. Best signal action is selected.
6. The result is displayed in browser output.

This modular design demonstrates how reinforcement learning systems can be packaged into deployable intelligent software.



System Architecture of Deployed Traffic Signal Control System

5.2 FLASK-BASED WEB APPLICATION INTERFACE

To provide practical accessibility, the reinforcement learning traffic controller was deployed as a Flask web application. Flask enables Python logic to be connected directly with HTML pages, making it suitable for lightweight machine learning deployment.

The web interface allows users to interact with the trained traffic controller through a browser. Instead of manually executing notebook code, users simply open the local application and enter traffic values into input fields.

The interface includes the following features:

INPUT FIELDS

Users enter the number of vehicles currently waiting in:

- North–South lane
- East–West lane

These values represent the current state of the traffic environment.

PREDICT DECISION BUTTON

After entering queue lengths, users submit the form. The application sends these values to the backend model for evaluation.

SIGNAL RECOMMENDATION OUTPUT

The system returns one of two decisions:

- Green Light for North–South
- Green Light for East–West

SIMPLE USER EXPERIENCE

The interface is intentionally lightweight and clear, allowing demonstrations during presentations or live testing.

The use of Flask provides several practical advantages:

- Easy deployment using Python
- Fast integration with trained RL models
- Browser-based access
- Lightweight resource usage
- Suitable for prototypes and academic demos

This deployment demonstrates that reinforcement learning models can be integrated into real software interfaces rather than remaining purely theoretical implementations.

5.3 REAL-TIME DECISION MAKING PROCESS

The deployed traffic controller performs dynamic decision making using the trained Q-learning policy. Unlike traditional fixed-time traffic lights, the reinforcement learning agent analyses traffic demand before selecting signal phases.

At each prediction request, the system receives the current queue state:

- Number of waiting vehicles in North–South direction
- Number of waiting vehicles in East–West direction

This state is used as input to the Q-table. The agent then compares the expected long-term reward of available actions.

Available actions include:

- Action 0: Allow East–West traffic
- Action 1: Allow North–South traffic

The action with the highest learned value is selected and returned as the signal recommendation.

For example:

If:

- North–South queue = 5
- East–West queue = 1

The model is likely to assign the green signal to North–South traffic because clearing the larger queue reduces congestion more efficiently.

Similarly, if East–West congestion becomes larger, the model changes its decision accordingly.

This process demonstrates adaptive control behaviour. The system does not rely on static timers but instead reacts to actual traffic demand.

Real-time decision capability is one of the strongest advantages of reinforcement learning in traffic management because traffic patterns constantly change throughout the day.

5.4 PRACTICAL VALIDATION OF THE DEPLOYED MODEL

After deployment, the model was tested using multiple traffic queue combinations to verify whether decisions remained logical and efficient outside the training environment.

Several scenarios were evaluated:

SCENARIO 1: BALANCED TRAFFIC

- North–South = 2
- East–West = 2

The system selected either direction fairly depending on learned tie-breaking behaviour.

SCENARIO 2: HEAVY NORTH–SOUTH CONGESTION

- North–South = 5
- East–West = 1

The model prioritised North–South traffic.

SCENARIO 3: HEAVY EAST–WEST CONGESTION

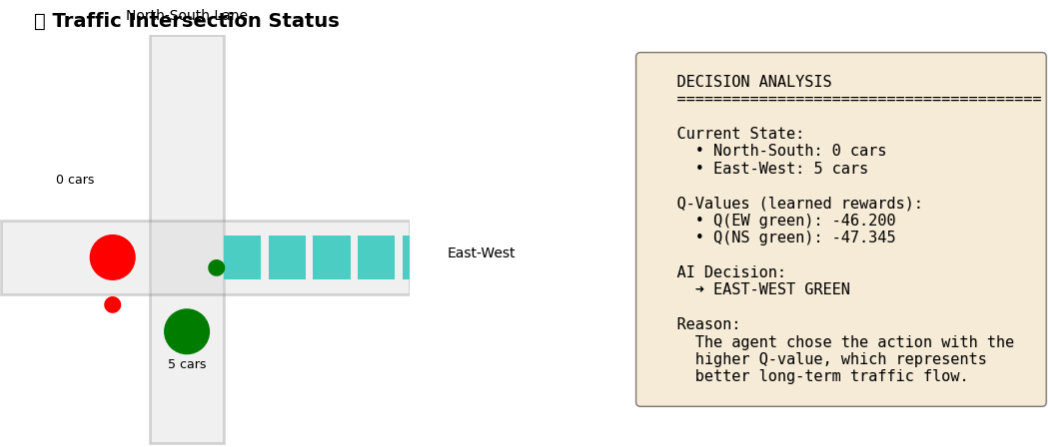
- North–South = 1
- East–West = 5

The model prioritised East–West traffic.

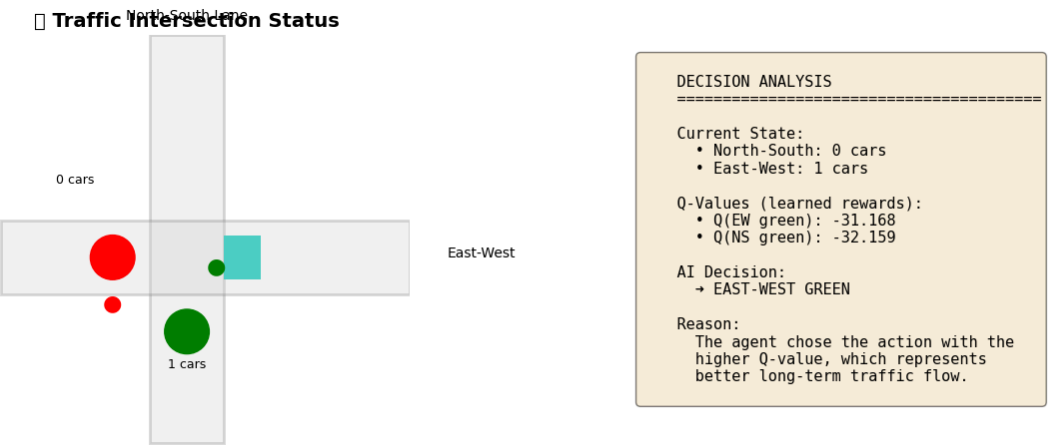
SCENARIO 4: REPEATED RANDOM INPUTS

Multiple random queue states were tested, and the model consistently selected decisions aligned with congestion reduction objectives.

These results validate that the deployed Q-learning model generalised effectively from training to practical use.



RL Decision Under Higher Congestion



RL Decision Under Low Traffic Demand

5.5 PERFORMANCE INTERPRETATION

The deployed system produced fast responses because Q-table lookup requires minimal computation. Unlike deep neural networks, tabular reinforcement learning allows immediate predictions.

Observed strengths include:

- Fast decision speed
- Adaptive signal allocation
- Reduced queue prioritisation
- Lightweight deployment
- Explainable actions through state-action table

The model performed best under traffic ranges similar to training states. Since the environment used limited queue capacities, extreme real-world traffic levels may require additional training.

Overall, performance indicates that reinforcement learning is effective for small-scale intelligent traffic control prototypes.

5.6 CHALLENGES BETWEEN THEORY AND DEPLOYMENT

Although the model performed well in simulation, several challenges were observed when converting theory into deployment.

LIMITED STATE SPACE

Training states used small queue ranges. Real traffic systems may contain much larger queues.

SIMPLIFIED ENVIRONMENT

The project considered only two directions and ignored turning lanes, pedestrians, and emergency vehicles.

MANUAL INPUT DEPENDENCY

Current deployment requires users to manually enter queue values rather than using live sensors or cameras.

STATIC LEARNED POLICY

The deployed Q-table does not continue learning automatically during usage.

These differences show that deployment introduces practical constraints beyond theoretical reinforcement learning experiments.

6 LIMITATIONS

Although the deployed Flask-based traffic controller demonstrates the practical use of reinforcement learning, several limitations exist due to the simplified nature of the prototype.

SIMPLIFIED INTERSECTION DESIGN

The current model considers only two traffic directions: North–South and East–West. Real intersections often contain turning lanes, pedestrian crossings, multiple lanes, roundabouts, and more complex signal phases. Therefore, the deployed system does not yet represent full urban traffic complexity.

LIMITED STATE SPACE

The Q-learning model was trained on a small number of traffic states based on queue lengths within restricted ranges. In real cities, queues may grow much larger and vary unpredictably. The current state space may not cover all real-world conditions.

MANUAL INPUT REQUIREMENT

The deployed Flask system depends on users manually entering queue lengths. This limits automation and realism. A practical intelligent traffic system would require live sensor feeds, cameras, or IoT devices to provide continuous traffic data.

STATIC POLICY AFTER TRAINING

The deployed model uses a trained Q-table and does not continue learning during operation. This means it cannot automatically adapt to new traffic patterns, roadworks, seasonal changes, or unexpected events unless retrained manually.

NO MULTI-INTERSECTION COORDINATION

Modern traffic systems often require multiple intersections to coordinate signals across a wider road network. The current deployment controls only one isolated junction and cannot optimise traffic flow citywide.

NO EMERGENCY VEHICLE PRIORITY

The prototype does not detect ambulances, fire engines, police vehicles, or buses. In real deployments, emergency vehicle priority is often essential for safety and public service efficiency.

BASIC USER INTERFACE

Although functional, the Flask interface is simple and mainly designed for demonstration. It lacks professional dashboards, traffic heatmaps, charts, admin controls, and reporting features expected in operational systems.

SIMULATION-REALITY GAP

The model was trained in a simulated environment with random traffic generation. Real traffic behaviour includes human unpredictability, accidents, weather effects, illegal driving, and special events. These factors are not captured in the current prototype.

COMPUTATIONAL SCALABILITY

Q-learning works well for small environments but becomes inefficient when state spaces grow larger. For large road networks, tabular methods may become impractical.

The current deployment is effective as a proof-of-concept prototype but would require substantial expansion, sensor integration, and advanced algorithms for real-world city deployment.

7 ETHICAL CONSIDERATIONS

The use of reinforcement learning for traffic signal control introduces several ethical considerations that should be addressed before any real-world deployment. Although intelligent traffic systems can improve efficiency and reduce congestion, they must also operate in a fair, safe, and transparent manner.

One of the most important ethical issues is fairness. If the system repeatedly prioritises one traffic direction over another, certain groups of road users may experience consistently longer delays. For example, residential roads or lower-volume routes could be disadvantaged if the model focuses only on maximising traffic flow on major roads. Therefore, fairness should be considered within the reward function and decision-making process.

Safety is another critical concern. Traffic lights directly influence vehicle movement, pedestrian crossings, and driver behaviour. Incorrect or delayed decisions made by an AI system could increase the risk of collisions, confusion, or dangerous braking events. For this reason, any real-world deployment would require strict safety constraints, fail-safe mechanisms, and the ability for human operators to override automated decisions when necessary.

Transparency and accountability are also important. Public authorities and road users should be able to understand how automated traffic decisions are made. If a system causes delays or unsafe behaviour, responsibility for those decisions must be clear. Reinforcement learning systems should therefore include logging, monitoring, and explainability features to support accountability.

Another ethical issue involves privacy. Future smart traffic systems may rely on cameras, sensors, GPS signals, or connected vehicle data. If such data is collected, it must comply with privacy regulations and avoid unnecessary storage of personal information. Data should be securely managed and used only for legitimate traffic management purposes.

Bias in training data is another potential risk. If the model is trained using traffic patterns from limited locations or time periods, it may not generalise fairly to other areas with different road usage patterns. Continuous evaluation and retraining would therefore be necessary.

Energy use and sustainability should also be considered. Although intelligent traffic systems can reduce idle time and emissions, large-scale AI infrastructure may require significant computing resources. Efficient system design would therefore be important.

Overall, responsible deployment of AI-based traffic control systems should balance efficiency with fairness, safety, privacy, and accountability. While the current project was conducted in simulation, these ethical considerations remain highly relevant for future real-world applications.

8 FUTURE WORK

Several improvements can be made to enhance the deployed traffic control system and move it closer to real-world smart city implementation.

Future versions can connect to cameras, inductive loop sensors, GPS traffic feeds, or IoT devices to automatically measure vehicle queues. This would eliminate manual data entry and enable fully automated operation. The project can be expanded from a single junction to multiple connected intersections. Reinforcement learning agents could coordinate signals across a road network to reduce citywide congestion rather than improving only one location. Instead of tabular Q-learning, future systems could use Deep Q-Networks (DQN), PPO, or Actor-Critic methods. These algorithms handle larger and continuous state spaces better, making them suitable for real traffic systems. The current model is static after training. A future version could continue learning from daily traffic data and adapt automatically to seasonal patterns, road closures, or changing commuter behaviour. The system could detect ambulances, fire trucks, or police vehicles and automatically prioritise green signals to improve emergency response times. Traffic lights could also prioritise buses or trams to improve public transport punctuality and reduce delays. Additional reward functions can be designed to prevent one lane from waiting too long. This would balance efficiency with fairness.

The Flask application can be upgraded into a professional dashboard showing:

- Live traffic maps
- Queue charts
- Historical reports
- Congestion analytics
- Manual override controls
- Alert notifications

The system could be hosted on cloud infrastructure for remote monitoring, scalability, and centralised management by transport authorities. Training with historical traffic datasets from cities would make the model more realistic and improve decision quality. Future reward functions could also minimise fuel wastage, carbon emissions, and idle engine time, supporting environmental goals. By combining reinforcement learning with sensors, deep learning, network coordination, and real infrastructure data, the current prototype could evolve into a powerful intelligent traffic management system suitable for smart cities.

9 CONCLUSION

The reinforcement learning traffic controller was successfully deployed as a Flask-based web application. The deployed model demonstrates that Q-learning can be transformed into a practical decision-making system capable of recommending traffic signals based on live queue conditions.

Testing confirmed that the system adapts its decisions according to congestion levels and responds quickly through a browser interface. Although simplified, the prototype provides strong evidence that reinforcement learning can support future intelligent traffic management systems.

The project successfully bridges the gap between reinforcement learning theory and software deployment, fulfilling the practical objectives of the assessment.

REFERENCES

- [1] Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [2] DeepMind, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [3] M. Wiering, “Multi-agent reinforcement learning for traffic light control,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2000, pp. 1151–1158.
- [4] T. Belletti et al., “Expert level control of ramp metering based on multi-task deep reinforcement learning,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 4, pp. 1198–1207, 2018.
- [5] H. Wei, G. Zheng, H. Yao, and Z. Li, “IntelliLight: A reinforcement learning approach for intelligent traffic light control,” in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2018, pp. 2496–2505.
- [6] Gymnasium Documentation. Farama Foundation.
- [7] Flask Documentation, “Flask Web Development Framework,” Pallets Project. Available: <https://flask.palletsprojects.com/>